분기한정 기법

현재까지의 비용만 봐도 더 좋은 답이 될 수 없으면 그 갈래를 멈춥니다.

먼저 작은 예로 살펴봅니다

분기한정

현재까지의 비용만 봐도 더 좋은 답이 될 수 없으면 그 갈래를 멈춥니다.

예: 이미 최저 비용 7을 찾음

다른 갈래의 현재 비용이 8이고 앞으로 비용을 더해야 한다면 멈춥니다.

8-1 정해진 순회 경로의 비용

구할 것

정해진 도시 방문 순서의 이동 비용을 더합니다.

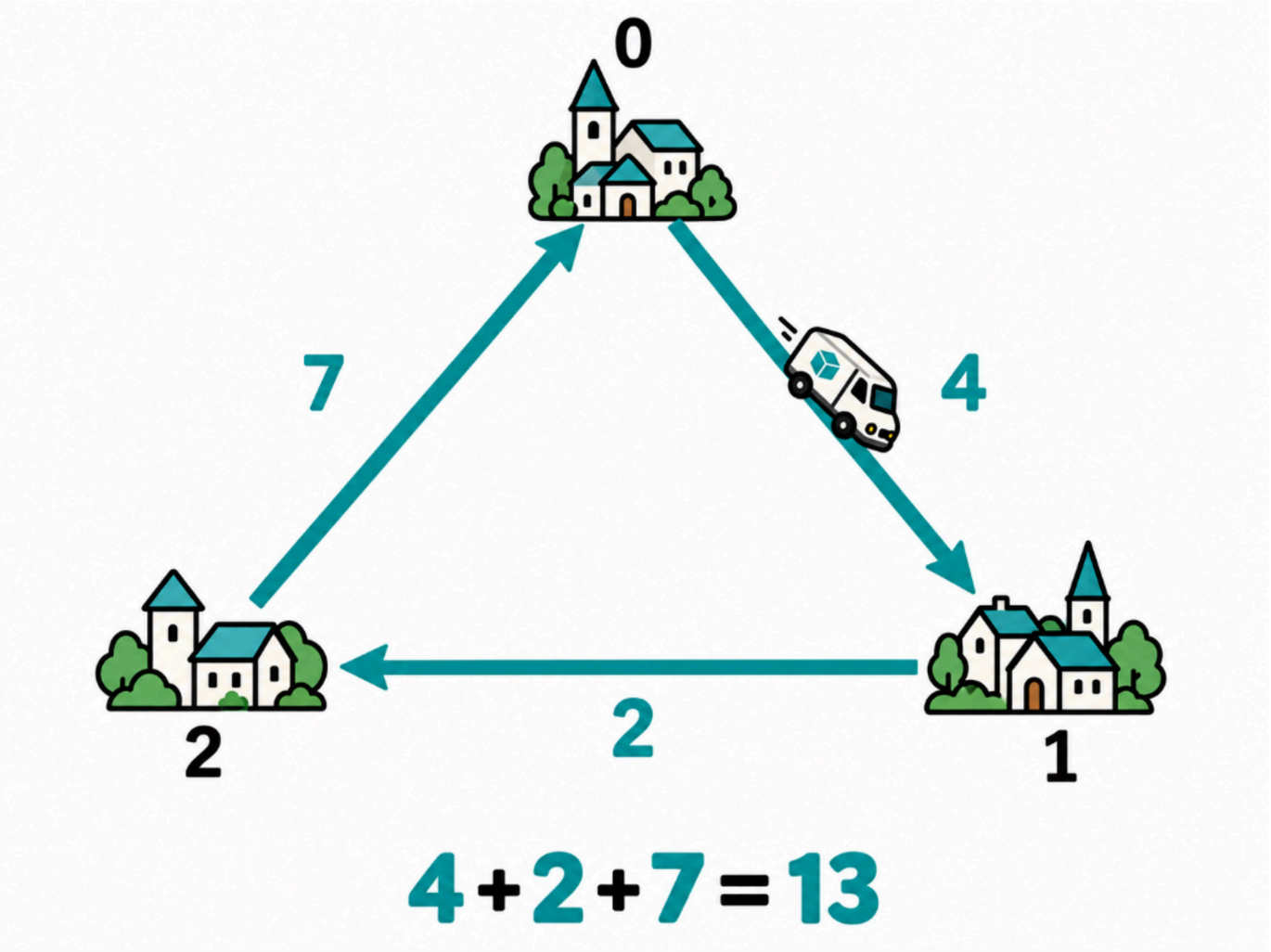
작은 예

0,1,2 순서로 방문합니다. 비용은 0→1이 4, 1→2가 2입니다.
마지막 2→0의 비용은 7입니다.

반환할 값

출발점으로 돌아오는 비용까지 4+2+7=13을 반환합니다.

8-1 그림으로 보는 예



0에서 1로: 4
1에서 2로: 2
2에서 0으로: 7
전체 비용은 13

이동 비용 표를 읽는 방법

0행 1열은 0번 도시에서 1번 도시로 가는 비용입니다.

출발 도시	도착0	도착1	도착2
도시0	0	4	7
도시1	4	0	2
도시2	7	2	0

예: dist[1][2]=2, dist[2][0]=7입니다.

8-1 하나씩 계산한 결과

이번 이동	더할 비용	지금까지 합
0에서 1	4	4
1에서 2	2	6
2에서 0	7	13

방문 순서는 입력으로 이미 정해져 있습니다. 그 순서의 비용만 계산합니다.

8-1 계산 순서

서로 이어진 도시

route[i]와 route[i+1]의 이동 비용을 더합니다.

마지막 도시

다음 도시를 출발점인 route[0]으로 둡니다.

비용 표 읽기

dist[출발][도착]에 적힌 값을 사용합니다.

8-1 변수와 코드의 뜻

코드	route=[0,1,2]일 때
i=0	dist[0][1]을 더함
i=1	dist[1][2]를 더함
i=2	dist[2][0]을 더함
(i+1)%3	다음 번호는 1,2,0. 마지막에 처음으로 돌아옴

행은 출발 도시, 열은 도착 도시입니다.

8-1 함수에 들어오는 값

함수

route_cost(dist, route)

입력 조건

도시 1~9개. 비용은 0~100, 대각선은 0. 경로는 0부터 모든 도시를 한 번씩 포함합니다.

8-1 Python

```
def route_cost(dist, route):
    total = 0
    for i in range(len(route)):
        total += dist[route[i]][route[(i+1)%len(route)]]
    return total
```

시간 O(n), 추가 공간 O(1)

8-1 C

```
int route_cost(const int dist[][9], int n, const int route[]) {
    int total=0;
    for (int i=0; i<n; ++i)
        total+=dist[route[i]][route[(i+1)%n]];
    return total;
}
```

시간 O(n), 추가 공간 O(1)

8-1 다른 입력도 확인합니다

입력	답	확인할 점
도시 하나	0	같은 도시로 돌아가는 비용 0
0→1은 5, 1→0은 8	13	방향에 따라 값이 달라도 됨
0,1,2 순서	13	마지막 귀환 포함

행은 출발 도시, 열은 도착 도시입니다.

8-2 남은 작업 비용의 하한

구할 것

남은 각 사람의 가장 싼 작업 비용을 더합니다.

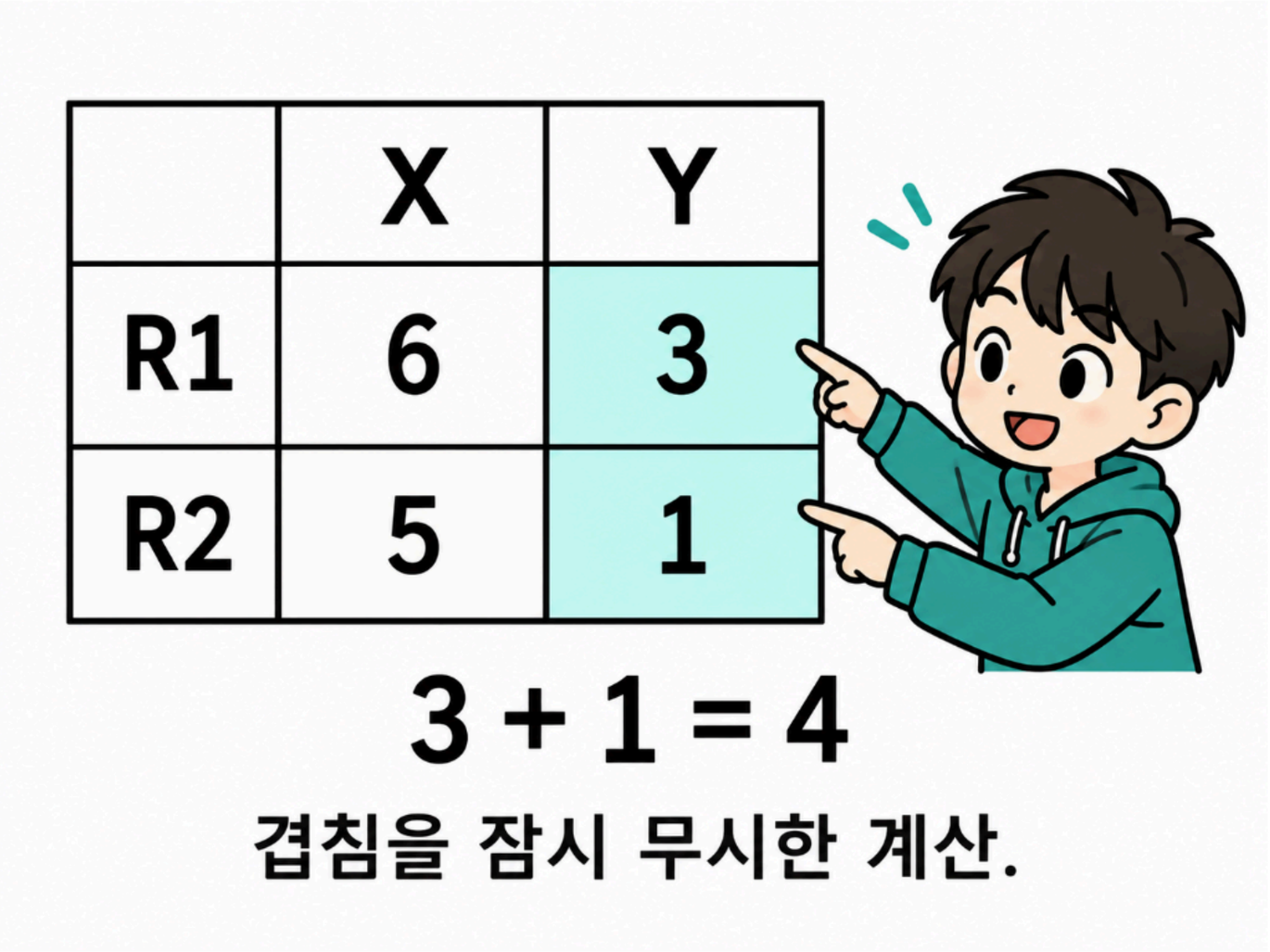
작은 예

0번 사람은 작업 1을 이미 맡았습니다.
남은 1번 사람의 최소 비용은 3, 2번 사람의 최소 비용은 1입니다.

반환할 값

이 계산에서는 같은 작업을 골라도 허용합니다. 3+1=4를 반환합니다.

8-2 그림으로 보는 예



그림의 X는 작업0
Y는 작업2입니다.
작업1은 이미 사용
각 행의 최소를 더해
3+1=4

8-2 하나씩 계산한 결과

사람	작업 0	작업 1	작업 2	고른 최소
0	9	2: 이미 배정	7	이번 계산에서 제외
1	6	4: 사용 불가	3	3
2	5	8: 사용 불가	1	1

서로 다른 사람에게 작업 2를 중복해서 골랐습니다. 비용을 낮게 잡기 위한 계산입니다.

왜 4가 완성 배정의 답은 아닐까요?

같은 작업을 두 사람에게 맡길 수 없기 때문입니다.

가능한 실제 배정	비용 합
1번 사람→작업0, 2번 사람→작업2	6+1=7
1번 사람→작업2, 2번 사람→작업0	3+5=8
실제 남은 최소 비용	7

낮게 잡은 계산은 4, 실제 가능한 최소 비용은 7입니다.

8-2 계산 순서

이미 쓴 작업 제외

used가 1인 열은 건너뜁니다.

남은 한 사람의 최소

그 행의 사용 가능한 열 중 가장 작은 값을 찾습니다.

행별 최소를 합산

남은 사람마다 반복해 합을 반환합니다.

8-2 변수와 코드의 뜻

변수	예제에서의 뜻
next_row=1	1번 사람부터 남아 있음
used=[0,1,0]	작업 1은 이미 사용, 0과 2는 사용 가능
low	이번 사람에게 가능한 가장 싼 비용
answer	사람별 low를 더한 값

이 값처럼 실제 최소 비용보다 크지 않게 잡은 값을 하한이라고 부릅니다.

8-2 함수에 들어오는 값

함수

assignment_bound(cost, used, next_row)

입력 조건

사람·작업 1~8개, 비용 0~100. used의 1 개수는 next_row와 같습니다.

8-2 Python

```
def assignment_bound(cost, used, next_row):
    n = len(cost)
    answer = 0
    for r in range(next_row, n):
        low = 10**9
        for c in range(n):
            if used[c] == 0:
                low = min(low, cost[r][c])
        answer += low
    return answer
```

시간 $O(n^2)$, 추가 공간 $O(1)$

8-2 C

```
int assignment_bound(const int cost[][8], int n,
    const int used[], int next_row) {
    int answer=0;
    for (int r=next_row; r<n; ++r) {
        int low=1000000;
        for (int c=0; c<n; ++c)
            if (!used[c] && cost[r][c]<low) low=cost[r][c];
        answer+=low;
    }
    return answer;
}
```

시간 $O(n^2)$, 추가 공간 $O(1)$

8-2 다른 입력도 확인합니다

상황	답	이유
남은 사람이 없음	0	더할 비용 없음
한 사람, 가능한 비용 7	7	최솟값 7
예제의 남은 두 사람	4	3+1

이 값처럼 실제 최소 비용보다 크지 않게 잡은 값을 하한이라고 부릅니다.

8-3 작업을 하나씩 배정하는 최소 비용

구할 것

각 사람에게 서로 다른 작업을 하나씩 맡겨 비용을 최소화합니다.

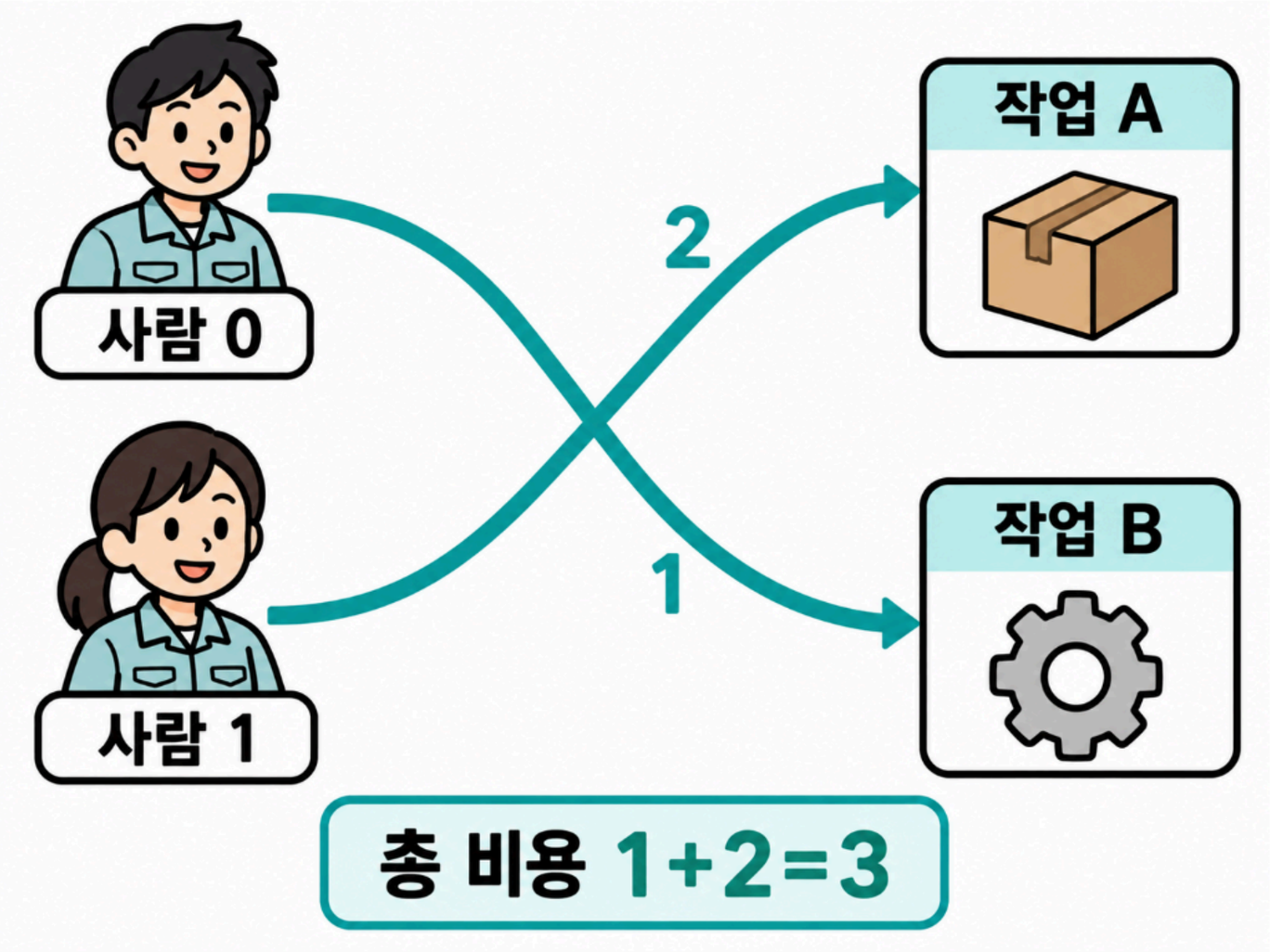
작은 예

사람 0의 비용은 [4,1], 사람 1의 비용은 [2,3]입니다.
작업 0과 1을 각각 한 번씩만 맡겨야 합니다.

반환할 값

사람0→작업1, 사람1→작업0이면 1+2=3입니다. 3을 반환합니다.

8-3 그림으로 보는 예



그림의 A는 작업0
B는 작업1입니다.
사람0에게 작업1
사람1에게 작업0
1+2=3

사람과 작업의 비용 표

행은 사람, 열은 작업입니다.

사람	작업0	작업1
사람0	4	1
사람1	2	3

사람0에게 작업1을 맡기면 1, 사람1에게 작업0을 맡기면 2입니다.

8-3 하나씩 계산한 결과

사람0의 작업	사람1의 작업	비용 합
0	1	4+3=7
1	0	1+2=3
최솟값		3

사람0과 사람1이 모두 작업1을 고르는 배정은 허용하지 않습니다.

현재 비용이 8이면 더 볼 필요가 없습니다

이미 비용 7짜리 완성 배정을 찾았다고 합시다.

남은 비용이 0이어도

현재 8+남은 0=8입니다. 이미 찾은 7보다 비쌉니다.

이 갈래의 처리

남은 작업을 전부 배정하기 전에 탐색을 중단합니다.

다음 갈래

방금 고른 작업의 사용 표시를 지우고 다른 작업을 시도합니다.

8-3 계산 순서

한 사람씩 배정

아직 사용하지 않은 작업을 하나 고릅니다.

완성한 배정의 비용 저장

지금까지 찾은 가장 싼 완성 비용을 best에 저장합니다.

이미 더 비싸면 중단

현재 비용이 best 이상이면 더 배정해도 좋아질 수 없어 돌아갑니다.

8-3 변수와 코드의 뜻

변수	뜻
row	이번에 작업을 맡길 사람 번호
used[c]	작업 c를 이미 맡겼으면 1
total	지금까지 배정한 비용의 합
best	이미 완성한 배정 중 가장 작은 비용

비용이 전부 0 이상이므로, 앞으로 작업을 더 맡겨도 total은 줄어들지 않습니다.

8-3 함수에 들어오는 값

함수

assignment_min(cost)

입력 조건

사람·작업 1~8개, 각 비용 0~100. 사람과 작업은 각각 한 번씩 배정합니다.

8-3 Python (1/2)

```
def assignment_min(cost):
    n = len(cost)
    used = [False]*n
    best = 10**9
    def search(row, total):
        nonlocal best
        if total >= best: return
        if row == n:
            best = total
```

다음 쪽에서 이어집니다.

8-3 Python (2/2)

```
            return
        for col in range(n):
            if not used[col]:
                used[col] = True
                search(row+1, total+cost[row][col])
                used[col] = False
    search(0, 0)
    return best
```

보수적 최악 시간 O(n·n!), 공간 O(n)

8-3 C

```
static int assign_go(const int cost[][8], int n, int row,
    int used[], int total, int best) {
    if (total>=best) return best;
    if (row==n) return total;
    for (int c=0; c<n; ++c) if (!used[c]) {
        used[c]=1;
        best=assign_go(cost,n,row+1,used,total+cost[row][c],best);
        used[c]=0;
    }
    return best;
}
int assignment_min(const int cost[][8], int n) {
    int used[8]={0};
    return assign_go(cost,n,0,used,0,1000000000);
}
```

보수적 최악 시간 O(n·n!), 공간 O(n)

8-3 다른 입력도 확인합니다

상황	답	확인할 점
한 사람, 비용 7	7	그 작업을 맡김
모든 비용 0	0	0도 정상 비용
2×2 예제	3	작업을 중복 사용하지 않음

비용이 전부 0 이상이므로, 앞으로 작업을 더 맡겨도 total은 줄어들지 않습니다.

8-4 모든 도시를 한 번씩 방문하는 최소 순회 비용

구할 것

도시를 모두 한 번씩 방문하고 출발점으로 돌아오는 최소 비용을 구합니다.

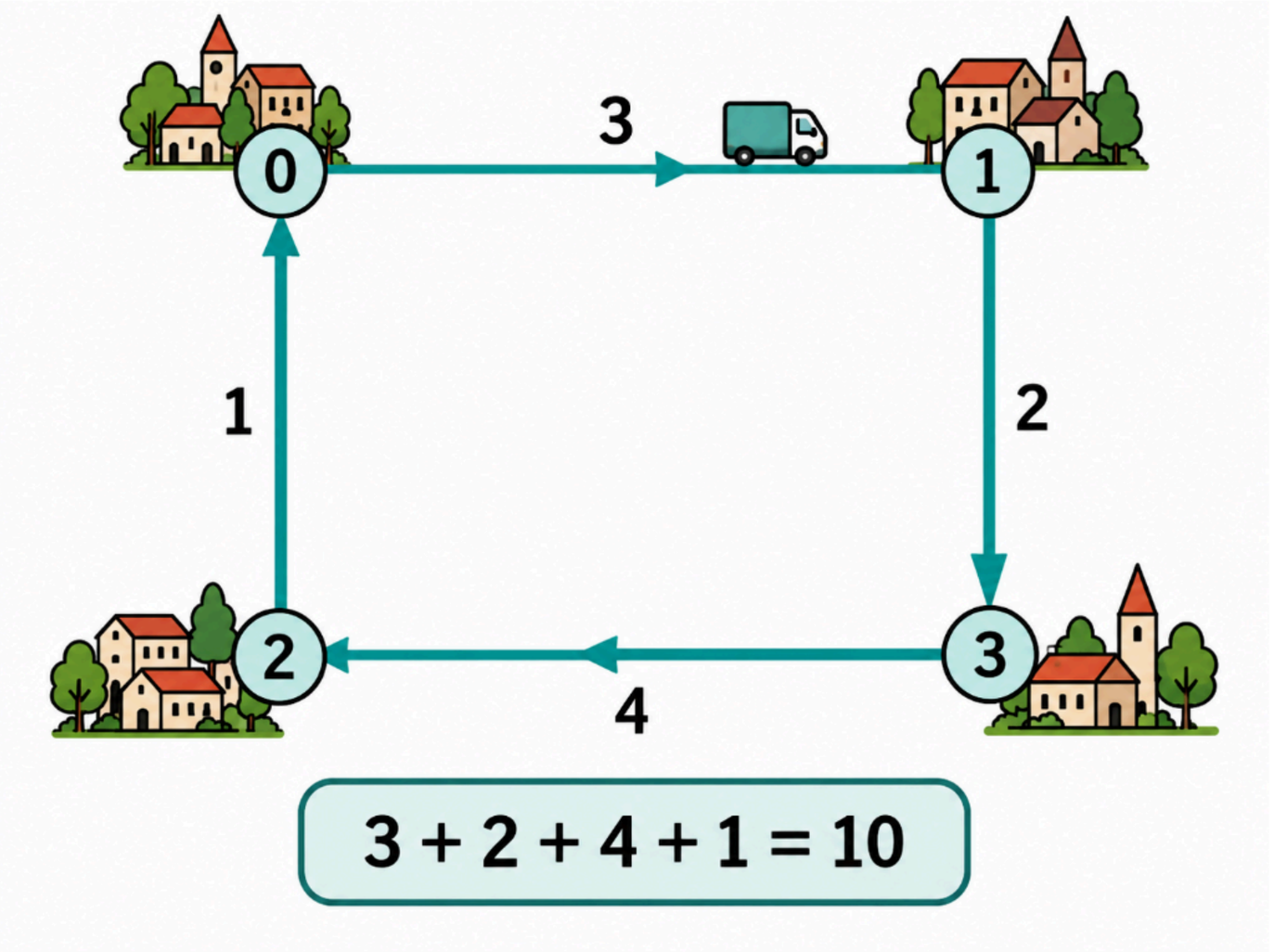
작은 예

네 도시의 방문 순서들을 비교합니다.
0,1,2,3,0은 비용 23이고, 0,1,3,2,0은 비용 10입니다.

반환할 값

가능한 순회 경로들의 비용 중 가장 작은 값을 반환합니다.

8-4 그림으로 보는 예



0,1,3,2,0 순서
3+2+4+1=10
다른 순회들도 비교해
최소 비용을 찾습니다.

네 도시 예제의 전체 비용 표

아래 표로 모든 순회의 비용을 계산할 수 있습니다.

출발	도착0	도착1	도착2	도착3
도시0	0	3	1	8
도시1	3	0	8	2
도시2	1	8	0	4
도시3	8	2	4	0

0,1,3,2,0의 비용은 3+2+4+1=10입니다.

8-4 하나씩 계산한 결과

0에서 출발하는 방문 순서	귀환 포함 비용
0,1,2,3,0	23
0,1,3,2,0	10
0,2,1,3,0	19
0,2,3,1,0	10
0,3,1,2,0	19
0,3,2,1,0	23

이 예의 최소 비용은 10입니다. 그림의 순회도 10입니다.

8-4 계산 순서

다음 도시 시도

아직 방문하지 않은 도시 하나로 갑니다.

도시를 모두 봤으면

마지막 도시에서 0으로 돌아오는 비용을 더합니다.

이미 더 비싸면 중단

현재까지의 비용이 best 이상이면 이 경로를 더 늘리지 않습니다.

8-4 변수와 코드의 뜻

변수	뜻
last	지금 서 있는 도시
count	0번을 포함해 지금까지 방문한 도시 수
total	현재까지 이동한 비용
used	방문한 도시 표시. 되돌아올 때 해제

8-1은 경로 하나의 비용, 8-4는 가능한 경로 중 최소 비용을 구합니다.

8-4 함수에 들어오는 값

함수

tsp_min(dist)

입력 조건

도시 1~9개. 비용 0~100, 대각선 0. 모든 이동이 가능하고 방향별 비용은 달라도 됩니다.

8-4 Python (1/2)

```
def tsp_min(dist):
    n = len(dist)
    used = [False]*n
    used[0] = True
    best = 10**9
    def search(last, count, total):
        nonlocal best
        if total >= best: return
        if count == n:
```

다음 쪽에서 이어집니다.

8-4 Python (2/2)

```
            best = min(best, total+dist[last][0])
            return
        for nxt in range(1, n):
            if not used[nxt]:
                used[nxt] = True
                search(nxt, count+1, total+dist[last][nxt])
                used[nxt] = False
    search(0, 1, 0)
    return best
```

보수적 최악 시간 O(n·(n−1)!), 공간 O(n)

8-4 C (1/2)

```
static int tour_go(const int d[][9], int n, int last,
    int count, int used[], int total, int best) {
    if (total>=best) return best;
    if (count==n) {
        int v=total+d[last][0];
        return v<best?v:best;
    }
    for (int x=1; x<n; ++x) if (!used[x]) {
        used[x]=1;
        best=tour_go(d,n,x,count+1,used,total+d[last][x],best);
```

다음 쪽에서 이어집니다.

8-4 C (2/2)

```
            used[x]=0;
        }
        return best;
}
int tsp_min(const int dist[][9], int n) {
        int used[9]={0};
        used[0]=1;
        return tour_go(dist,n,0,1,used,0,1000000000);
}
```

보수적 최악 시간 O(n·(n−1)!), 공간 O(n)

8-4 다른 입력도 확인합니다

상황	답	확인할 점
도시 하나	0	이동 비용 없음
두 도시, 왕복 5와 8	13	귀환 비용 포함
위의 네 도시 예제	10	가능한 여섯 순회 중 최소

8-1은 경로 하나의 비용, 8-4는 가능한 경로 중 최소 비용을 구합니다.

8장의 계산을 한 문장으로

문제	구하는 값
8-1	정해진 도시 방문 순서의 이동 비용을 더합니다.
8-2	남은 각 사람의 가장 싼 작업 비용을 더합니다.
8-3	각 사람에게 서로 다른 작업을 하나씩 맡겨 비용을 최소화합니다.
8-4	도시를 모두 한 번씩 방문하고 출발점으로 돌아오는 최소 비용을 구합니다.

현재까지의 비용만 봐도 더 좋은 답이 될 수 없으면 그 갈래를 멈춥니다.